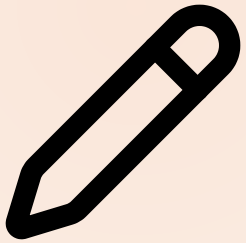
MODULE 02 · 24 MIN

Prompting Like a Tech Lead



A great prompt is a spec. Write it the way a Tech Lead writes a ticket.

Theory · The GCOE prompt (4 min)

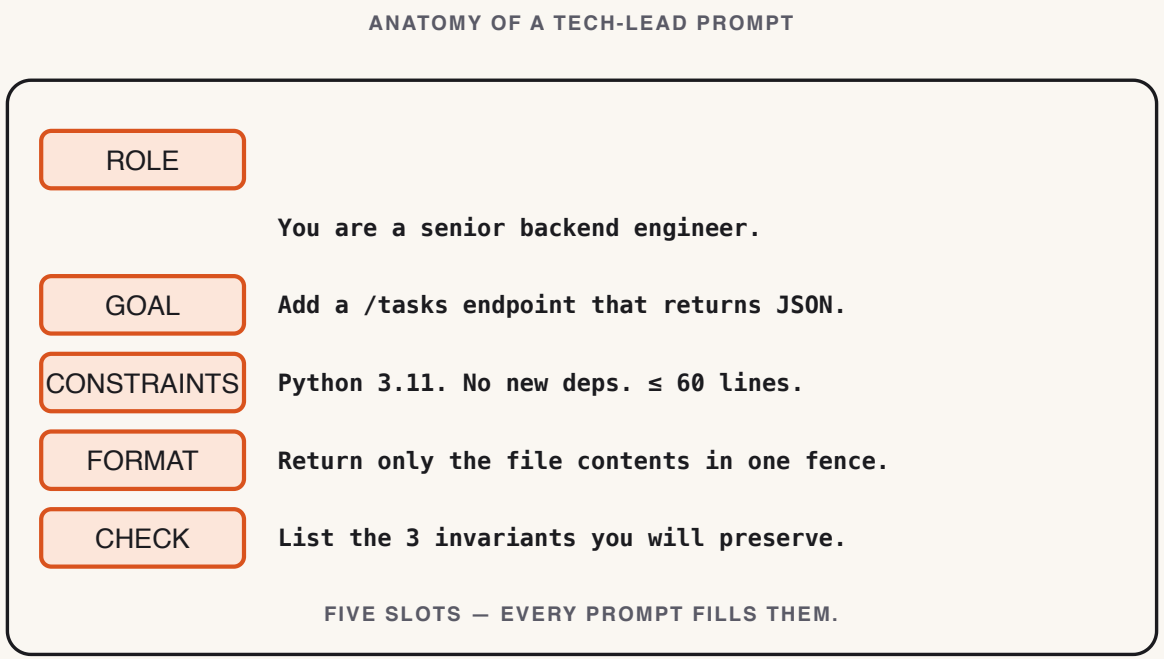
A production prompt has **four parts — skip one and quality drops**:

> **G**oal · **C**onstraints · **O**utput format · **E**xamples

- **Goal** — one verb-led sentence: what can the user *do* at the end?
- **Constraints** — language, deps, file layout, error handling, and what must **not** happen.
- **Output format** — files, exit codes, JSON shapes if any.
- **Examples** — one happy path + one edge case is enough to disambiguate.

A vague prompt produces plausible code that fails review. GCOE produces code you can merge.

Anatomy of a GCOE prompt



Goal · **C**onstraints · **O**utput · **E**xamples — skip one and quality drops.

Reference · GCOE skeleton you can paste

```
GOAL: A user can <verb> <thing> from the command line.

CONSTRAINTS:
- Language: Python 3.11, standard library only (no third-party deps).
- Persist state to ./tasks.json.
- Exit codes: 0 success, 1 user error, 2 internal error.

OUTPUT:
- A single runnable script + a short README with usage.

EXAMPLES:
- `task add "Buy milk"` -> prints new id, exit 0.
- `task done 999` (missing id) -> prints error to stderr, exit 1.
```

Keep it tight. Every line removes one wrong guess Claude could make.

Reference · Common mistakes

- "Build a CLI" with no constraints — looks fine, fails review.
- Allowing unintended third-party deps (the constraint exists for a reason).
- Skipping examples and exit codes — production CLIs are graded on exit codes, not stdout.

Live demo · Vague vs. GCOE (5 min)

Step 1 — paste the vague prompt:

```
Make a CLI to manage tasks.
```

Step 2 — paste the GCOE prompt and re-run:

```
GOAL: A user can add, list, complete, and delete tasks from the command line.
CONSTRAINTS: Python 3.11, stdlib only; persist to ./tasks.json;
  exit codes 0 success / 1 user error / 2 internal error.
OUTPUT: one runnable script + a short usage README.
EXAMPLES: `task add "Buy milk"` -> prints id, exit 0;
  `task done 999` -> stderr error, exit 1.
```

Success signal: the GCOE version runs all four commands with correct exit codes; the vague one doesn't.

Your turn · CLI Task Manager (12 min)

Exercise: `exercises/part-02/README.md`

Build a CLI with four commands, persisted to `tasks.json`:

```
task add "<title>"        # -> prints new id
task list [--status STATE]
task done <id>
task delete <id>
```

Prompt: start from the GCOE skeleton; fill Goal/Constraints/Output/Examples for *your* task manager.

Deliverable: working CLI in `module-02/` + `iteration-notes.md` recording one deleted constraint and its code diff.

Success signal: all four commands run end-to-end; exit codes are `0` / `1` / `2`.

✓ Done & next (1 min)

Definition of done

- [ ] Four commands work; `tasks.json` round-trips state.
- [ ] Exit codes correct (`0` / `1` / `2`).
- [ ] `iteration-notes.md` documents one deleted constraint + diff.

Next — a good prompt is per-task. Now we make Claude follow your repo's rules *automatically*.

Module 3 — Project Context with CLAUDE.md.